

Document number	P3211R1
Date	2024-07-04
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

views::flat_map

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Implementation experience
 - Proposed change
 - References

Abstract

We propose `views::flat_map`, a range adaptor that applies a function returning a range for each element, then flattens the result. This pattern, commonly known as *flat mapping*, is widespread in functional programming and data processing. Providing it as a dedicated view improves readability and expressiveness, and also opens opportunities for optimization in lazy evaluation contexts.

Noted that this is ranked as Tier 1 in [P2760](#).

Revision history

R0

Initial revision.

R1

Rename `views::transform_join` to `views::flat_map`.

Introduce new `flat_map_view` class.

Discuss related optimization.

Discussion

Mapping each element to a subrange and flattening the results into a single range is common in programming tasks like data processing, string handling, and range composition.

This appears frequently enough in practice to justify direct support in the standard ranges library. Providing `views::flat_map` encourages clearer, more maintainable code and lays the groundwork for potential *optimizations* specific to this use case. While the same behavior can be achieved through composition such as `views::transform` and then `views::join`, a dedicated view enables the library to make stronger semantic guarantees and allows more efficient handling of transform function results, particularly for expensive or lvalue-producing mappers.

This pattern arises naturally in code like:

```
auto all_courses = students
    | std::views::flat_map([](const Student& s) {
        return std::views::all(s.courses);
    });
```

Design

Why `views::flat_map` instead of `views::transform_join`?

The name `transform_join` describes how to combine `transform` and `join`, but it does not show the real concept. Users may see it as two steps, not as a single *map and flatten* operation; The name `flat_map` is common in many languages and clearly shows both mapping and flattening. This makes it easier to find, less confusing, and matches what users expect.

The following table shows how this operation is named in various languages, all of which converge on *flat-map*:

Language	Function Name	Example	Differences in Semantics
Haskell	<code>concatMap (>=)</code>	<code>concatMap f [1, 2, 3]</code>	Purely functional; <code>>=</code> is monadic bind, generalizing flat mapping beyond lists
C# (LINQ)	<code>SelectMany</code>	<code>source.SelectMany(x => ...)</code>	Applies to all <code>IEnumerable</code> ; default for flattening nested queries in LINQ syntax
Python	<code>itertools.chain.from_iterable(map(...))</code>	<code>list(chain.from_iterable(map(f, data)))</code>	No built-in <code>flatMap</code> ; semantics are manual; relies on strict evaluation and eager lists
Java	<code>flatMap</code>	<code>stream.flatMap(f)</code>	Requires the mapper function to return a <code>Stream</code> ; lazy evaluation is enforced
JavaScript	<code>flatMap</code>	<code>array.flatMap(x => [x, x + 1])</code>	One-level flattening only; not lazy; limited to arrays
Kotlin	<code>flatMap</code>	<code>list.flatMap { listOf(it, it * 2) }</code>	Similar to Java, but cleaner syntax; strict (not lazy) evaluation for standard collections
Rust	<code>flat_map</code>	<code>iter.flat_map(x some_iter(x))</code>	Applies to iterators; lazy by default; consumes the original iterator
Swift	<code>flatMap</code>	<code>array.flatMap { [\$0, \$0 * 2] }</code>	Semantics changed in Swift 4 - used to also remove optionals; now strictly flatten + map

Given this widespread usage, `flat_map` is the most appropriate and intuitive name for the proposed view. It reflects established terminology, avoids over-specifying implementation details, and aligns well with programmer expectations.

It is worth noting that the `range/v3` uses the name `views::for_each` for this operation. Because such naming deviates further from common terminology and could cause additional confusion, it is not considered a suitable option.

Additionally, C++23 introduced a container named `std::flat_map`, which is an associative container that stores key-value pairs. This is conceptually quite different from the proposed `views::flat_map`, which is a range adaptor that composes a map-then-flatten operation. Clarifying this distinction helps avoid confusion between the two.

Finally, while `flat_map` is preferred for its clarity and alignment with established usage, other naming alternatives are acceptable as long as they convey the correct semantics.

Why shouldn't be `views::transform(f) | views::join`?

Although `flat_map` can be expressed as a composition of `transform` and `join`, a dedicated `flat_map_view` provides clearer semantics and more efficient behavior in practice.

The first minor issue with composition is that it interferes with `base()`. When writing `transform(f) | join`, the result holds a `transform_view` internally, so calling `base()` does not give access to the original range, which can be surprising and inconvenient in generic contexts.

Second, a standalone view can manage both the outer iterator and the inner range. It can cache the transform result and avoid *repeated* calls, which is important when the function returns an lvalue range.

In the composed form, `join_view` has no knowledge of how the inner range was produced - it treats each element as if it were obtained by dereferencing the outer iterator. This works fine when the base is something like vector of vectors, where dereferencing is cheap and deterministic. But with `transform | join`, each inner range is the result of invoking a user-defined transformation, which could be expensive to compute even if the result is a stable lvalue. For example:

```
auto outer = views::iota(1, 5);
auto inner = views::iota(1, 10);
auto flatten = views::flat_map(outer, [&](int i) -> auto& { return inner; });
println("{} ", flatten);           // calls transform function 40 times
```

While `cache_latest` can mitigate redundant evaluation, it forces the result to model only an `input_range`, regardless of the actual capabilities of the underlying ranges. It also adds an extra layer of adaptor composition and complexity. In contrast, a dedicated `flat_map_view` can handle this caching internally while preserving the strongest valid iterator category.

A dedicated view also avoids unnecessary template instantiations. Since a flat-mapped range can never be more than bidirectional, there is no benefit in preserving random-access capabilities through `transform_view`. However, when using adaptor composition, those capabilities may still be instantiated - even though `join_view` will downgrade them - leading to code bloat and slower compilation.

In short, using composition is flexible but less efficient and can be awkward. `flat_map_view` avoids repeated evaluations, keeps the best iterator type, and makes the view stack simpler.

Although paper [P2760](#) suggests there is little benefit to providing a separate view as state: *"Importantly, there really isn't much benefit to providing a bespoke `transform_join` as opposed to simply implementing it in terms of these two existing adaptors."*, practical usage has revealed that the composed form introduces subtle inefficiencies, especially in the presence of expensive transformation functions, and not easy to eliminate without internal knowledge of how adaptors interact.

How to store inner ranges

When the inner range is a reference, we can avoid repeated invocations of the mapping function by storing a pointer to the result instead. This allows us to simply dereference the pointer whenever the inner range is needed, rather than invoking the transformation again — a technique somewhat analogous to `cache_latest_view`. However, it's important to note that this pointer must be stored inside the iterator, not the `flat_map_view` itself, in order to ensure correct behavior in multi-pass scenarios where multiple iterators may coexist and advance independently.

For prvalue inner ranges, the situation is somewhat more tricky, especially for *immovable* ranges.

For `join_view`, its underlying range's iterator already returns the inner range. This property makes it directly suitable for *non-propagating-cache::emplace-deref*, which caches the inner range by dereferencing the outer iterator.

In contrast, `flat_map_view` obtains the inner range by applying a mapping function to the element referenced by the outer iterator, rather than by directly dereferencing the outer iterator to yield the inner range. As a result, it cannot straightforwardly use *emplace-deref* to cache the inner range. Instead, it must store the entire inner range object returned by the mapping function within it to properly extend its lifetime.

One possible approach to mitigate this is to define a proxy iterator that models invoking the mapping function upon dereference. This proxy can then be used with *emplace-deref* to cache the mapped inner range indirectly, albeit with additional complexity.

Support for stashing flattening

At first glance, `flat_map_view` might not have a stashing issue as discussed in [P2770](#), since its inner range is always produced by applying a mapping function, rather than being a subrange stored within the outer range.

However, in particular, the mapping function can certainly return a range whose lifetime is tied to the outer iterator - for example, applying `views::split('x')` to strings obtained from an `istream_iterator<string>` — then `flat_map_view` needs to cache the current outer iterator to ensure the inner range remains valid during iteration. Therefore, `flat_map_view` follows the design pattern of that paper.

Implementation experience

The author implemented `views::flat_map` based on libstdc++, see [here](#).

The implementation supports `input_range`, `forward_range`, and `bidirectional_range`, and demonstrates correct caching behavior for both lvalue and prvalue immovable mapped ranges.

Proposed change

This wording is relative to [Latest Working Draft](#).

1. Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define cpp_lib_ranges_flat_map_2025XXL // freestanding, also in <ranges>
```

2. Modify 25.2 [\[ranges.syn\]](#), Header `<ranges>` synopsis, as indicated:

```
// mostly freestanding
#include <compare>           // see \[compare.syn\]
#include <initializer_list>  // see \[initializer.list.syn\]
#include <iterator>          // see \[iterator.synopsis\]

namespace std::ranges {
    [...]
    namespace views { inline constexpr unspecified join_with = unspecified; }

    // \[range.flat.map\], flat map view
    template<input_range V, move_constructible F>
    requires see below
    class flat_map_view;

    namespace views { inline constexpr unspecified flat_map = unspecified; }
    [...]
}
```

3. Add **25.7.? Flat map view** [\[range.flat.map\]](#) after 25.7.15 [\[range.join.with\]](#) as indicated:

-1- A flat-map view maps each element to a range and flattens the results into a view.

-2- The name `views::flat_map` denotes a range adaptor object ([\[range.adaptor.object\]](#)). Given subexpressions `E` and `F`, the expression `views::flat_map(E, F)` is expression-equivalent to `flat_map_view(E, F)`.

-3- *[Example 1:*

```
vector ints{0, 1, 2};
for (auto elem : ints | views::flat_map([](int i) { return views::repeat(i, 3); }))
    cout << elem << ' '; // prints 0 0 0 1 1 1 2 2 2
```

— *end example]*

[25.7.?2] Class template `flat_map_view` [\[range.flat.map\]](#)

```
namespace std::ranges {
    template<class R, class F>
    concept flat_mappable = input_range<R> &&                                // exposition only
                           regular_invocable<F&, range_reference_t<R>>> &&
                           input_range<invoke_result_t<F&, range_reference_t<R>>>;

    template<input_range V, move_constructible F>
    requires view<V> && is_object_v<F> && flat_mappable<V, F>
    class flat_map_view : public view_interface<flat_map_view<V, F>> {
    private:
        using MappedRng = invoke_result_t<F&, range_reference_t<V>>; // exposition only

        // \[range.flat.map.iterator\], class template flat_map_view::iterator
        template<bool Const>
        struct iterator; // exposition only

        // \[range.flat.map.sentinel\], class template flat_map_view::sentinel
        template<bool Const>
        struct sentinel; // exposition only

        V base_ = V(); // exposition only
        movable_box<F> fun_; // exposition only

        non_propagating_cache<iterator_t<V>>> current_; // exposition only, present only
                                                         // when !forward_range<V>
        non_propagating_cache<MappedRng> mapped_; // exposition only, present only
                                                         // if is_reference_v<MappedRng> is false
    public:
        flat_map_view() requires default_initializable<V> && default_initializable<F> = default;
        constexpr explicit flat_map_view(V base, F fun);
```



```

constexpr iterator(Parent& parent, BaseIter current)
    requires forward_range<Base>; // exposition only
constexpr explicit iterator(Parent& parent)
    requires (!forward_range<Base>); // exposition only

public:
using iterator_concept = see below;
using iterator_category = see below; // not always present
using value_type = range_value_t<MappedBase>;
using difference_type = see below;

iterator() = default;

template<class MappedRng = invoke_result_t<F&, range_reference_t<V>>>
constexpr iterator(iterator<!Const> i)
    requires Const &&
        is_reference_v<MappedRng> &&
        convertible_to<iterator_t<V>, BaseIter> &&
        convertible_to<iterator_t<MappedRng>, MappedIter> &&
        convertible_to<add_pointer_t<MappedRng>, add_pointer_t<MappedBase>>;

constexpr decltype(auto) operator*() const { return **mapped_it_; }

constexpr MappedIter operator->() const
    requires has_arrow<MappedIter> && copyable<MappedIter>;

constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int)
    requires mapped_is_glvalue && forward_range<Base> &&
        forward_range<MappedBase>;

constexpr iterator& operator--()
    requires mapped_is_glvalue && bidirectional_range<Base> &&
        bidirectional_common<MappedBase>;

constexpr iterator operator--(int)
    requires mapped_is_glvalue && bidirectional_range<Base> &&
        bidirectional_common<MappedBase>;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires mapped_is_glvalue && forward_range<Base> &&
        equality_comparable<iterator_t<MappedBase>>;

friend constexpr decltype(auto) iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(*i.mapped_it_))) {
    return ranges::iter_move(*i.mapped_it_);
}

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(*x.mapped_it_, *y.mapped_it_)))
    requires indirectly_swappable<MappedIter>;
};
}

```

-1- `iterator::iterator_concept` is defined as follows:

- (1.1) — If `mapped_is_glvalue` is true, `Base` models `bidirectional_range`, and `MappedBase` models `bidirectional_common`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- (1.2) — Otherwise, if `mapped_is_glvalue` is true and `Base` and `MappedBase` each model `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.
- (1.3) — Otherwise, `iterator_concept` denotes `input_iterator_tag`.

-2- The member `typedef-name iterator_category` is defined if and only if `mapped_is_glvalue` is true, `Base` models `forward_range`, and `MappedBase` models `forward_range`. In that case, `iterator::iterator_category` is defined as follows:

- (2.1) — Let `BASEC` denote `iterator_traits<iterator_t<Base>>::iterator_category`, and let `MAPPEDC` denote `iterator_traits<iterator_t<MappedBase>>::iterator_category`.
- (2.2) — If `BASEC` and `MAPPEDC` each model `derived_from<bidirectional_iterator_tag>` and `MappedBase` models `common_range`, `iterator_category` denotes `bidirectional_iterator_tag`.
- (2.3) — Otherwise, if `BASEC` and `MAPPEDC` each model `derived_from<forward_iterator_tag>`, `iterator_category` denotes `forward_iterator_tag`.

(2.4) — Otherwise, `iterator_category` denotes `input_iterator_tag`.

-3- `iterator::difference_type` denotes the type:

```
common_type_t<range_difference_t<Base>, range_difference_t<MappedBase>>
```

-4- `flat_map_view` iterators use the `satisfy` function to skip over empty mapped ranges.

```
constexpr BaseIter& current();  
constexpr const BaseIter& current() const;
```

-5- Returns: `current_` if `Base` models `forward_range`; otherwise, `*parent_>current_`.

```
constexpr auto& update-mapped();
```

-6- Effects:

(6.1) — If `mapped-is-glvalue` is `true`, equivalent to:

```
mapped_ = addressof(as-lvalue(std::invoke(*parent_>fun_, *current())));  
return *mapped_;
```

(6.1) — Otherwise, equivalent to:

```
return parent_>mapped_.emplace-def(map-iterator(*parent_>fun_, current()));
```

where `map-iterator` is the exposition-only class:

```
class map-iterator {  
    maybe-const<Const, F>& fun_;  
    const BaseIter& current_;  
    constexpr map-iterator(maybe-const<Const, F>& fun, const BaseIter& current)  
        : fun_(fun), current_(current) {}  
public:  
    constexpr auto operator*() const noexcept(noexcept(std::invoke(fun_, *current_))) {  
        return std::invoke(fun_, *current_);  
    }  
};
```

```
constexpr auto& get-mapped();
```

-7- Effects: Equivalent to:

```
if constexpr (mapped-is-glvalue)  
    return *mapped_;  
else  
    return *parent_>mapped_;
```

```
constexpr void satisfy();
```

-8- Effects: Equivalent to:

```
for (; current() != ranges::end(parent_>base_); ++current()) {  
    auto& mapped = update-mapped();  
    mapped_it_ = ranges::begin(mapped);  
    if (*mapped_it_ != ranges::end(mapped))  
        return;  
}  
if constexpr (mapped-is-glvalue)  
    mapped_it_.reset();
```

```
constexpr iterator(Parent& parent, BaseIter current)  
    requires forward_range<Base>;
```

-9- Effects: Initializes `current_` with `std::move(current)` and `parent_` with `addressof(parent)`; then calls `satisfy()`.

```
constexpr explicit iterator(Parent& parent)  
    requires (!forward_range<Base>;
```

-10- Effects: Initializes `parent_` with `addressof(parent)`; then calls `satisfy()`.

```
template<class MappedRng = invoke_result_t<F&, range_reference_t<V>>>  
    constexpr iterator(iterator<!Const> i)
```



```

requires Const &&
    is_reference_v<MappedRng> &&
    convertible_to<iterator_t<V>, BaseIter> &&
    convertible_to<iterator_t<MappedRng>, MappedIter> &&
    convertible_to<add_pointer_t<MappedRng>, add_pointer_t<MappedBase>>;

-11- Effects: Initializes current_ with std::move(i.current_), mapped_it_ with std::move(i.mapped_it_),
parent_ with i.parent_, and mapped_ with i.mapped_.

-12- [Note 1: Const can only be true when Base models forward_range. — end note]

constexpr MappedIter operator->() const
    requires has_arrow<MappedIter> && copyable<MappedIter>;

-13- Effects: Equivalent to: return *mapped_it_;

constexpr iterator& operator++();

-14- Effects: Equivalent to:

    if (++*mapped_it_ == ranges::end(get_mapped())) {
        ++current();
        satisfy();
    }
    return *this;

constexpr void operator++(int);

-15- Effects: Equivalent to: ++*this.

constexpr iterator operator++(int)
    requires mapped_is_glvalue && forward_range<Base> &&
    forward_range<MappedBase>;

-16- Effects: Equivalent to:

    auto tmp = *this;
    ++*this;
    return tmp;

constexpr iterator& operator--()
    requires mapped_is_glvalue && bidirectional_range<Base> &&
    bidirectional_common<MappedBase>;

-17- Effects: Equivalent to:

    if (current_ == ranges::end(parent_>base_)) {
        --current_;
        mapped_it_ = ranges::end(update_mapped());
    }

    while (mapped_it_ == ranges::begin(get_mapped())) {
        --current_;
        mapped_it_ = ranges::end(update_mapped());
    }
    --*mapped_it_;
    return *this;

constexpr iterator operator--(int)
    requires mapped_is_glvalue && bidirectional_range<Base> &&
    bidirectional_common<MappedBase>;

-18- Effects: Equivalent to:

    auto tmp = *this;
    --*this;
    return tmp;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires mapped_is_glvalue && forward_range<Base> &&
    equality_comparable<iterator_t<MappedBase>>;

-19- Effects: Equivalent to: return x.current_ == y.current_ && x.mapped_it_ == y.mapped_it_;

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(*x.mapped_it_, *y.mapped_it_)))

```


requires indirectly_swappable<MappedIter>;

-20- *Effects*: Equivalent to: `ranges::iter_swap(*x.mapped_it_, *y.mapped_it_)`;

[25.7.7.3] Class template `flat_map_view::sentinel` [range.flat.map.sentinel]

```
namespace std::ranges {
    template<input_range V, move_constructible F>
        requires view<V> && is_object_v<F> && flat_mappable<V, F>
        template<bool Const>
        struct flat_map_view<V, F::sentinel {
        private:
            using Parent = maybe_const<Const, flat_map_view>;           // exposition only
            using Base = maybe_const<Const, V>;                         // exposition only
            sentinel_t<Base> end_ = sentinel_t<Base>();                 // exposition only

        public:
            sentinel() = default;

            constexpr explicit sentinel(Parent& parent);
            constexpr sentinel(sentinel<!Const> s)
                requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

            template<bool OtherConst>
                requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
            friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
        };
    }
```

constexpr explicit sentinel(Parent& parent);

-1- *Effects*: Initializes `end_` with `ranges::end(parent.base_)`.

```
constexpr sentinel(sentinel<!Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

-2- *Effects*: Initializes `end_` with `std::move(s.end_)`.

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

-3- *Effects*: Equivalent to: `return x.current() == y.end_;`

References

[P2760R1]

Barry Revzin. A Plan for C++26 Ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html>

[P2770R0]

Tim Song. Stashing stashing iterators for proper flattening. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2770r0.html>